

Advanced Computer Networks



Dr Sudipta Saha

Associate Professor

Dept of Computer Science & Engineering
Indian Institute of Technology Bhubaneswar

Raj Jain's Theory



DSSRG:
Decentralized Smart
Systems Research Group

<https://sites.google.com/iitbbs.ac.in/dssrg>

Or Google dssrg iitbbs



Root Cause of Congestion

- **Huge Heterogeneity** in the technology

Fiber optical Cable

Twisted Pairs

Wireless communication

→ Mismatch in the **Arrival Rate** and **Service Rates** -
in the intermediate nodes

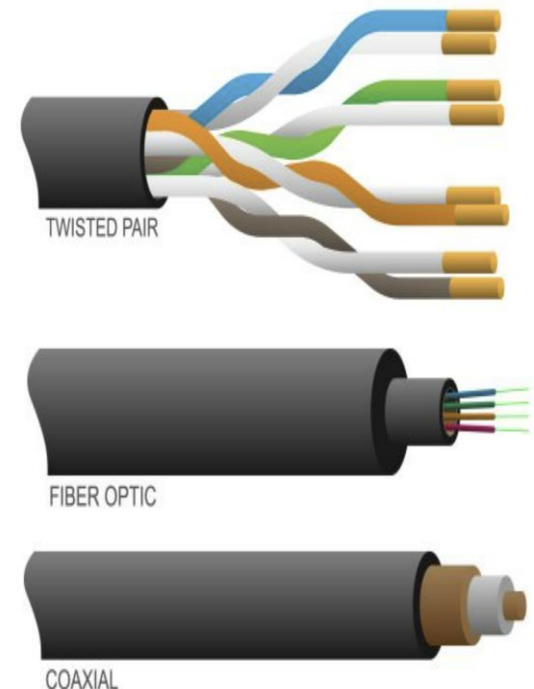
→ **Increased queuing and congestion**

We are concerned about -

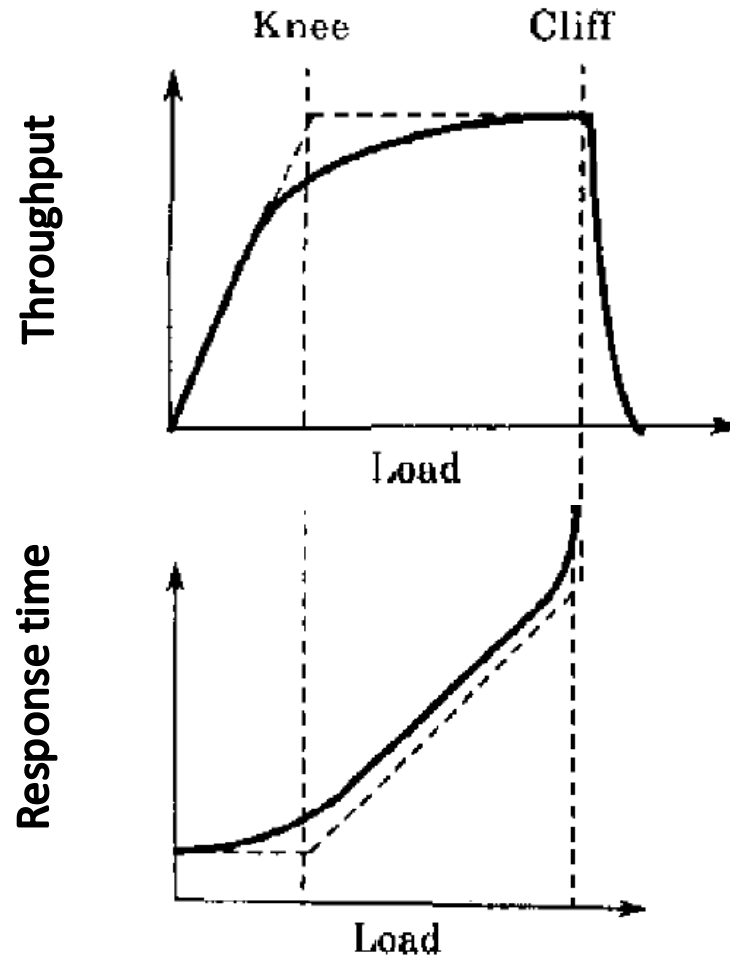
Load, Throughput and Response time

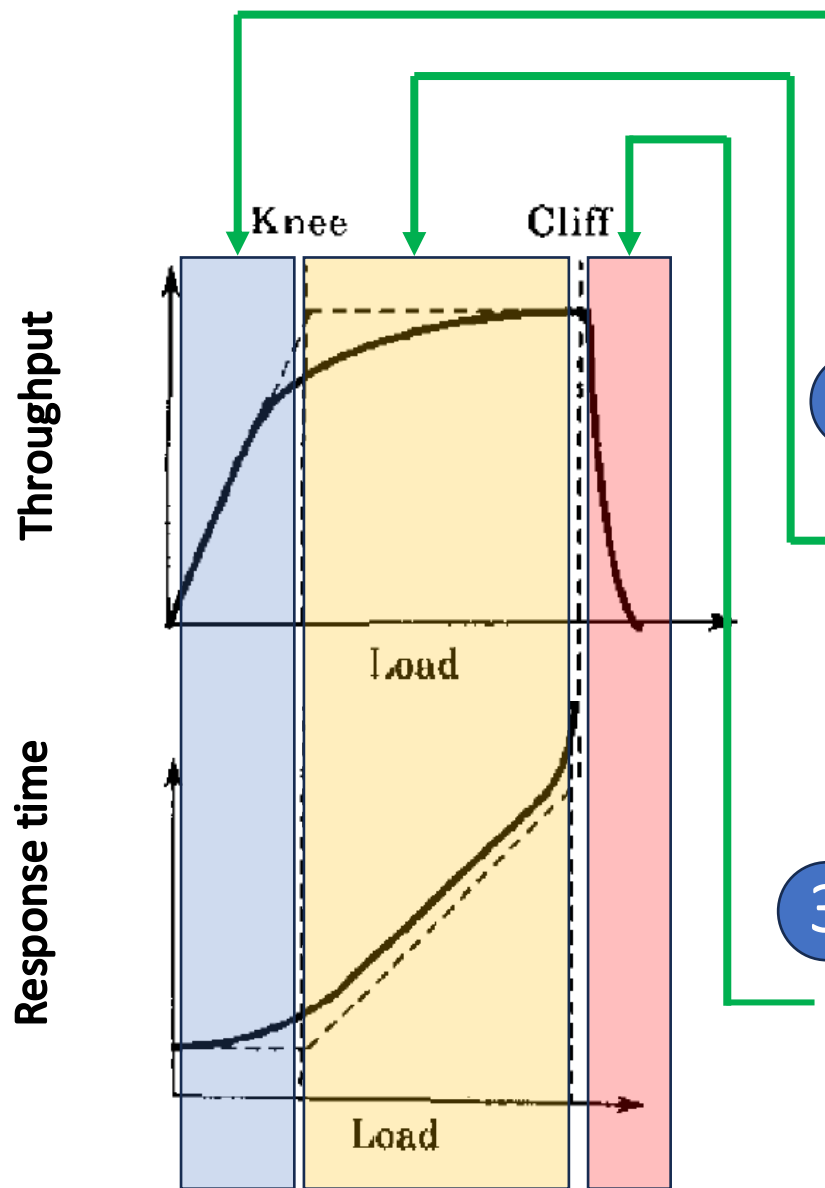
What is the general characteristics of

- **Load vs throughput** and
- **Load vs response time**



Throughput Vs Load and Response Time Vs Load





• Small Load: [Until knee]

1

- No significant queue buildup
- **Throughput:** Keeps up with the load
- **Response Time:** Very little increase

2

• Increasing Load [Until Cliff]

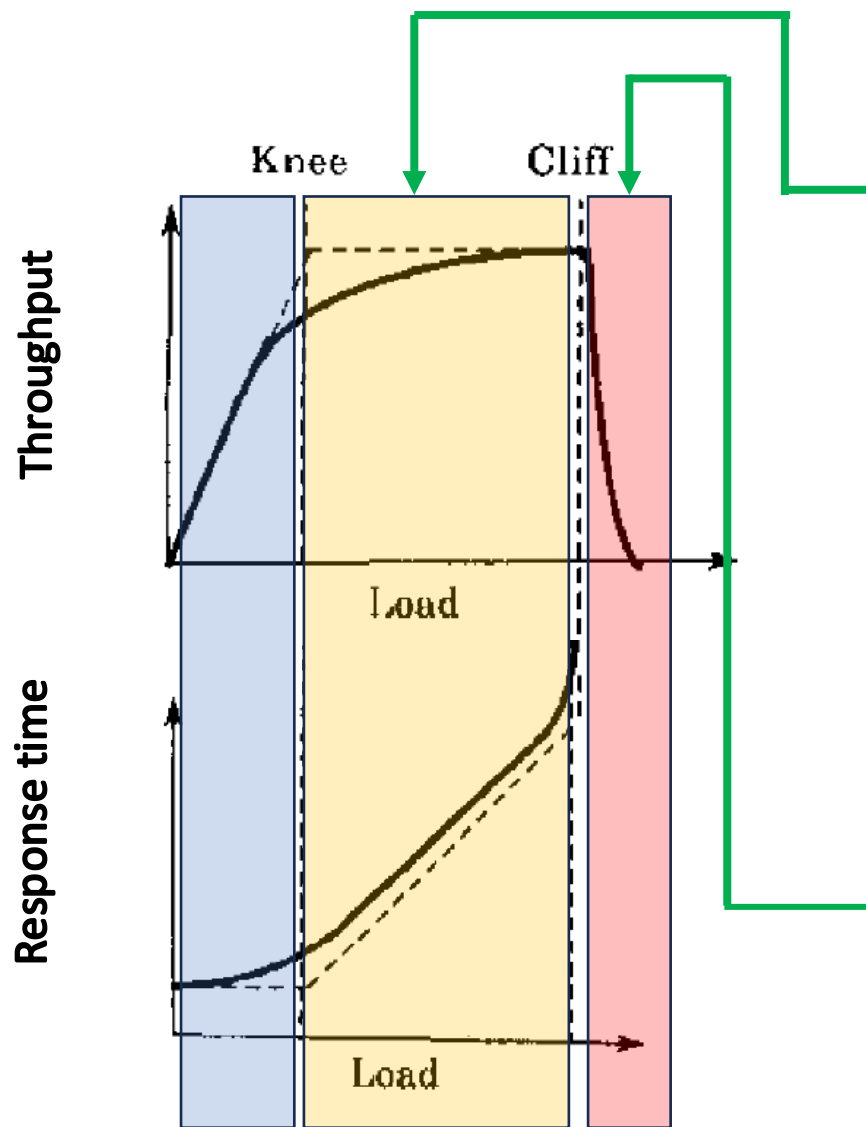
- Queue starts building up
- Response Time: Increases Linearly

3

• Excessive load

- Queue starts overflowing
- Throughput drops
- Response times shoots up

Congestion control and avoidance



Knee:

Increase in throughput is small
Significant increase in response time

Congestion avoidance scheme
enables the system to operate at knee
It will encourage traffic until it
significantly affects response time

Oscillation around the knee

After Cliff:

Packets start getting lost
Throughput falls fast

Congestion control scheme enables
a system to operate before cliff
and after knee

Control and Avoidance

- Dynamic Resource Management Problem
 - Basic steps -
 - *System senses its state*
 - *Feed this back to the users*
 - *Users adjust their controls*
- Any congestion control:
Increase or decrease
mechanism**
- State of a system – Total load on the network
 - Control of a system - Number of packets put into the network
 - How to limit: **Window size / Rate per user**
 - Can be adjusted based on the total load of the system



Binary Feedback Mechanism

- Resources in the network monitor their usage
- Determine if they are loaded below or above an optimal load level
- The resource sends a binary feedback to the users
 - 1 = Overloaded
 - 0 = Underloaded
- The users adjust their loads using increase and decrease algorithm
 - Setting a bit in the packet header
 - Connectionless networking protocol standards
 - Called Congestion Experience Bit
 - Quality of Service in the network layer header



Distributed System

- All users of the same bottleneck resource will get the same feedback
- Users adjust so that bottleneck is efficiently used and Equally shared by all the users
- We assume synchronous feedback and control loop
 - All users receive the same feedback
 - All users react to it
 - Next feedback is generated after all users reacted
 - Process is repeated



Centralized Vs Decentralized

- We consider one bottleneck resource and its users
- Distributed algorithm for managing distributed resource
- Centralized vs Decentralized decision making



Centralized decision

**Information flows to
the resource manager**

**Decision is made by
the resource**

Decentralized decision

**Information flows to the
users by the resource**

**Decision is made by the
users**

The level of feedback is a very important issue.
We consider a binary feedback



Why binary?

- **Simplicity**
- Easy to generate the feedback for the resource
- Quick generation of the feedback
- Simple measures to be taken by the users and hence fast



Model

- **n** users sharing a network
- Congestion in the system depends on the no of packets
- Discrete time operation → Time divided into small slots

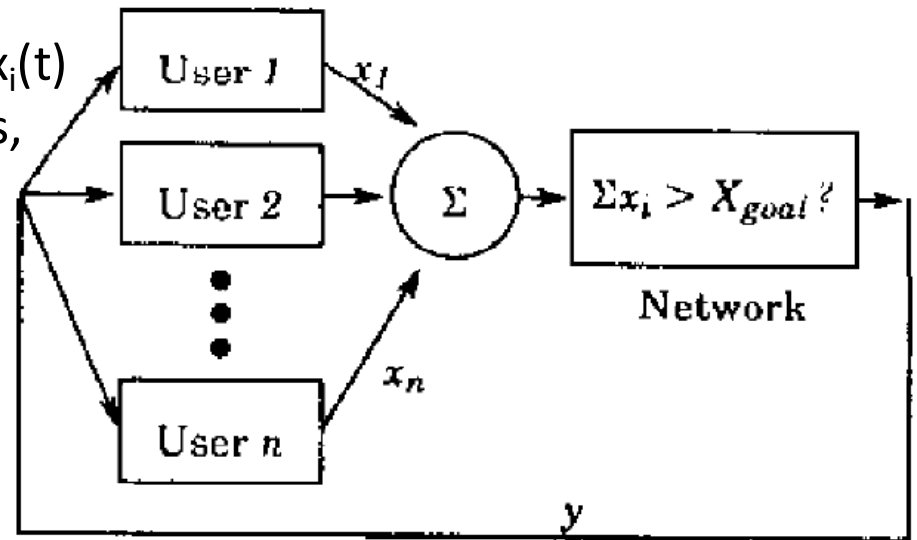
Users set their load based on previous feedback In the previous interval

Consider time slot t , i -th user's load is $x_i(t)$
Total load at the bottleneck user is thus,

$$\sum_{i=1}^n x_i(t)$$

System state $x(t) =$

$$\{x_1(t), x_2(t), x_3(t) \dots \dots, x_n(t)\}$$



Control

- $x_i(t)$ denotes the demand & allocated resource to i -th user
- **The target is to operate at the knee**
- During the interval the system calculates a load status and send a binary feedback to the users say $y(t)$
- Increase load $y(t) = 0$
- Decrease load $y(t) = 1$



Control

- Users based on this signal changes their demand by an amount $u_i(t)$ – it represents i-th users control

$$x_i(t+1) = x_i(t) + u(t)$$

- It is a function of the **users previous demand and the system's feedback**
- So $u_i(t) = f(x_i(t), y(t))$
- Combinedly $x_i(t+1) = x_i(t) + f(x_i(t), y(t))$



Control

- Ideally $u(t)$ should depend on the other users individual demand – But this is not possible
- f can be any **linear or nonlinear function** – for now we will consider **linear functions only** -
- **For increasing, when $y = 0$, $x_i(t+1) = a_I + b_I \cdot x_i(t)$**
- **For decreasing, when $y = 1$, $x_i(t+1) = a_D + b_D \cdot x_i(t)$**
- **Why linear? (of the form $y = mx + c$)**
[a_I, a_D, b_I, b_D are appropriately decided constants]



Let us see the possibilities --

- Multiplicative Increase and Multiplicative Decrease (MIMD)
- Additive Increase and Additive Decrease (AIAD)
- Additive Increase and Multiplicative Decrease (AIMD)
- Multiplicative Increase and Additive Decrease (MIMD)



Functional forms ...

Multiplicative Increase and Multiplicative Decrease (MIMD)

- For increasing, when $y = 0$,
- $x_i(t+1) = b_I \cdot x_i(t)$
- For decreasing, when $y = 1$,
- $x_i(t+1) = b_D \cdot x_i(t)$
- Load increases or decreases based on b_D
So, $b_I > 1$ and $0 < b_D < 1$



Possible simple linear functions

Additive Increase and Additive Decrease (AIAD)

- For increasing, when $y = 0$,
- $x_i(t+1) = a_I + x_i(t)$
- For decreasing, when $y = 1$,
- $x_i(t+1) = a_D + x_i(t)$
- Load increases or decreases based on a_I and a_D

So, $a_I > 0$ and $a_D < 0$



Possible simple linear functions

Additive Increase and Multiplicative Decrease (AIMD)

- For increasing, when $y = 0$, $x_i(t+1) = a_I + x_i(t)$
- For decreasing, when $y = 1$, $x_i(t+1) = b_D \cdot x_i(t)$
- Load increases or decreases based on a_I and b_D

So, $a_I > 1$ and $0 < b_D < 1$



Possible simple linear functions

Multiplicative Increase and Additive Decrease (MIAD)

- For increasing, when $y = 0$, $x_i(t+1) = b_I \cdot x_i(t)$
- For decreasing, when $y = 1$, $x_i(t+1) = a_D + x_i(t)$
- Load increases or decreases based on b_I and a_D

So, $b_I > 1$ and $a_D < 0$



Criteria for selection of the control

- We need to understand which is better and why

(1) Efficiency:

How well the resource is being utilized?

(2) Fairness:

Is the resource being well divided among the users?

(3) Distributedness:

How well the scheme works under minimal feedback?

(4) Convergence:

Does the scheme converges? How fast it does?



Efficiency (*Utilization of the resource*)

- *How well the resource is being utilized OR How close the total load is to the maximum load that the system can support*

Note:

It does not care what each user faces

- Let us assume X_{goal} = Desired load level at Knee
- $X_t = \sum x_i(t)$: This is the total load from all the users
- System is operating “efficiently” as long as the X_{goal} is close to X_t

Two allocations:
 $X_1 > X_2$ OR $X_1 < X_2 \rightarrow$

May have same **efficiency** as long as total load is close to knee

- $X_t > X_{goal}$, System is **overloaded**
 - $X_t < X_{goal}$, System is **underloaded**
- Both are undesirable and inefficient**



Fairness (Users' shares)

Consider **N** users are sharing **M** different resources

Different users may have different resource as their bottleneck resource

Set of users are partitioned into equivalence classes based on which resource has become its bottleneck

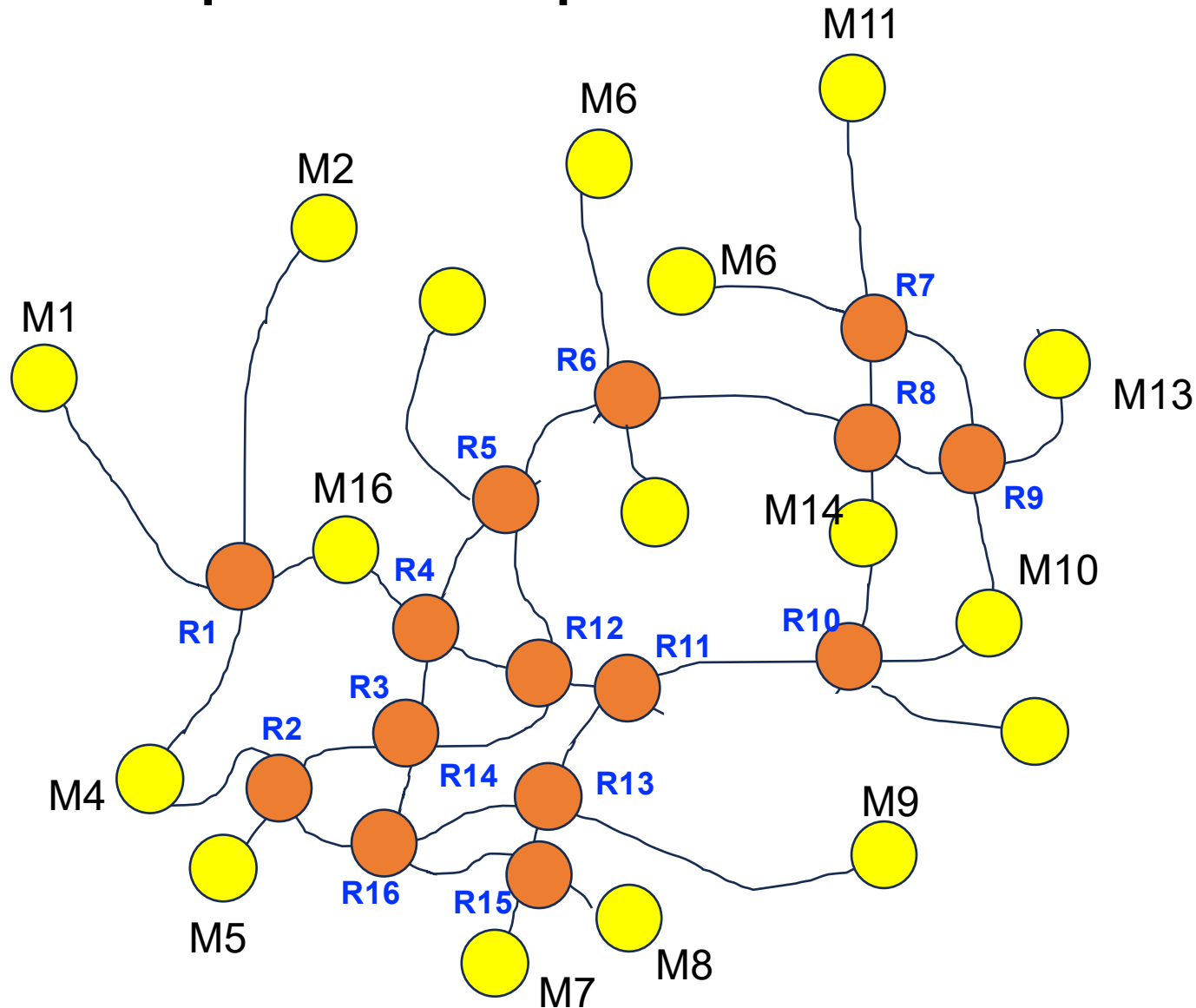
Ideally, Users under a *single equivalence class*

→ All the users should have equal share of the corresponding bottleneck resource

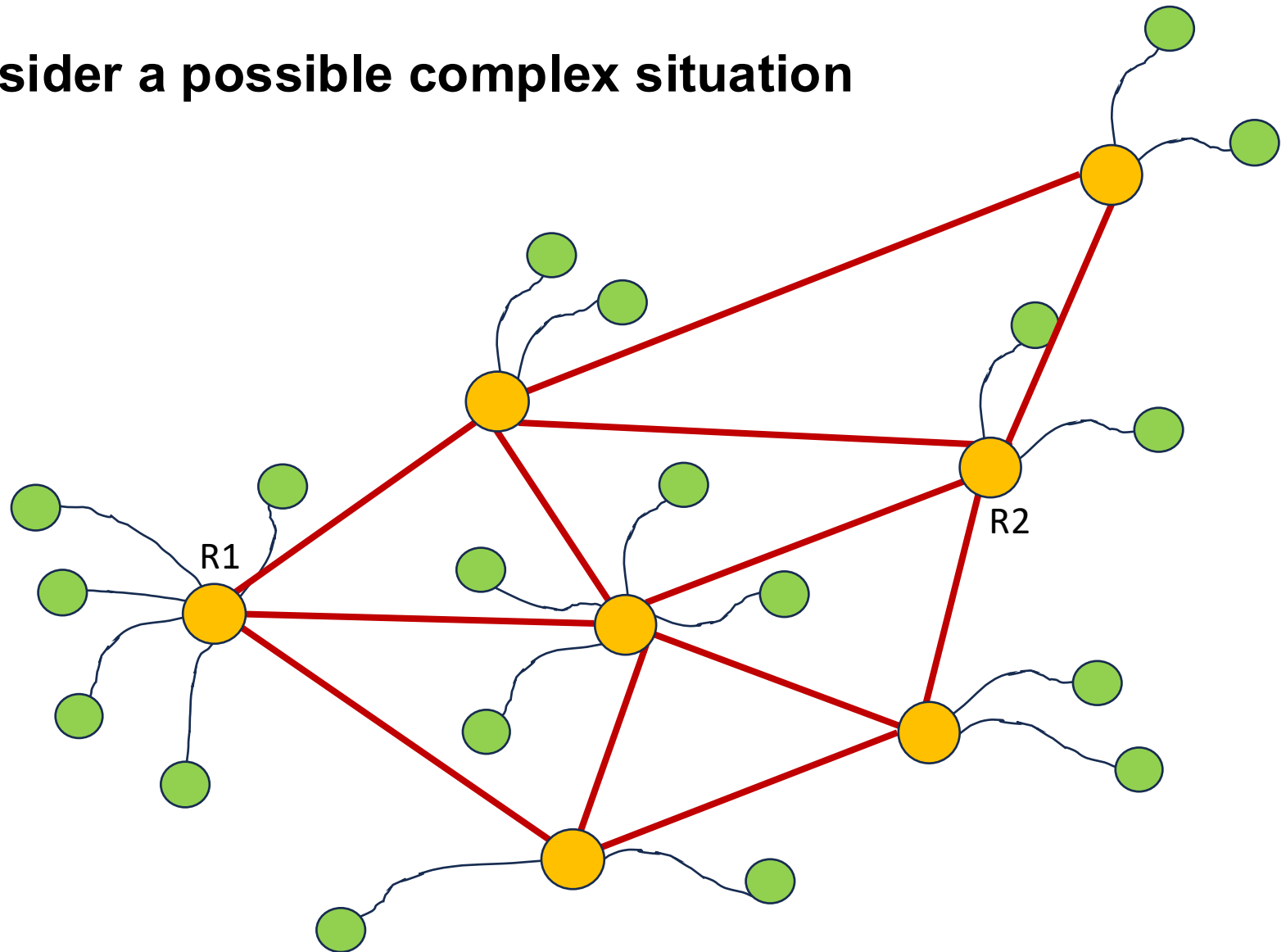
$$x_i(t) = x_j(t), \text{ for all pairs } (i,j) \text{ of users} \\ \text{in single equivalence class}$$



Consider a possible complex situation



Consider a possible complex situation



Fairness

- Less fair when all users do not have same share
- How to get a numerical value → a metric is needed
- A possible one: $F = \frac{(\sum x(t))^2}{n \sum (x(t))^2}$
- This is fundamentally - sum's square / (n*squares' sum)
- **Example:**
 - Consider a value 18 – divided into three users in 3 ways
 - Way 1 = **6,6,6** → Fairness = $F1 = (6+6+6)^2 / [3 * (6^2 + 6^2 + 6^2)] = 1$
 - Way 2 = **5,6,7** → Fairness = $F2 = (5+6+7)^2 / [3 * (5^2 + 6^2 + 7^2)] = 0.98$
 - Way 3 = **0,0,18** → Fairness = $F3 = (0+0+18)^2 / [3 * (0^2 + 0^2 + 18^2)] = 1/3 = 0.33$



Fairness (Properties)

So, the function gives

- 1 when all get equal share
 - **Maximum fairness**
- $1/n$ when one user gets the full share and rest 0
 - **Worst fairness**
- Fairness criteria is independent of scale (Units)
- Slight change in the allocation **will show up in F**
- If only k of n users share the resource equally –
We get k/n as the fairness value



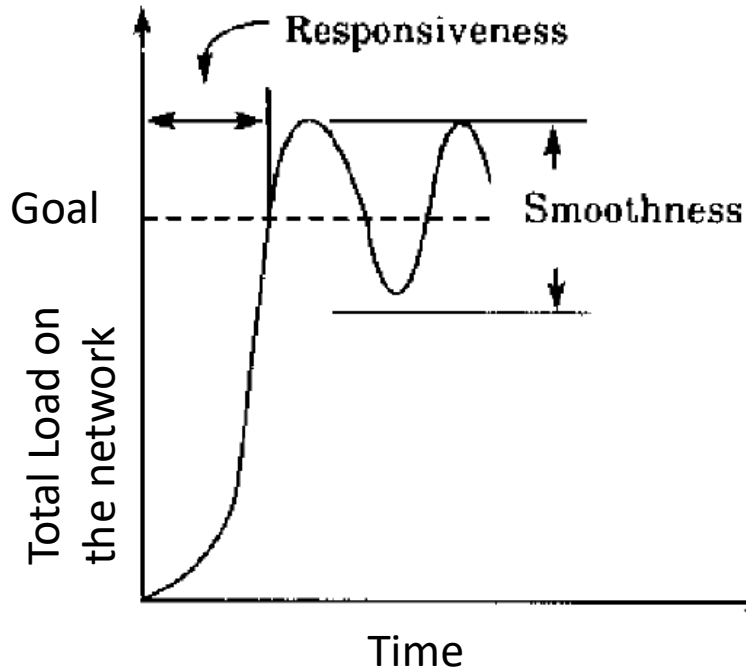
We use this function.

Distributedness

- Centralized strategy needs a complete knowledge of the system – e.g., **Each individual users demand and the sum**
- We have these information at the resource
- But conveying these information to each users is a critical issue [why?]
- Users may be using several resources at the same time → Additional use of the networks
- Thus, the scheme should work with minimum amount of feedback from the network. Simple information should work, e.g., ***Underloaded (0) or Overloaded (1)***



Convergence



Responsiveness as fast as possible and Oscillation should be as small as possible respectively:

These two determines Convergence

- A scheme needs to converge
- Time taken to approach the goal from any starting state
- But feedback is binary → So, it does not converge to a single steady state
- Rather - system reaches an equilibrium where it oscillates around the optimal state
- Responsiveness:
Time taken to reach equilibrium
- Smoothness:
The size of the oscillation

- Apart from this we have to meet –
- ***Simplicity, Robustness and Scalability !!!***



Linear Control

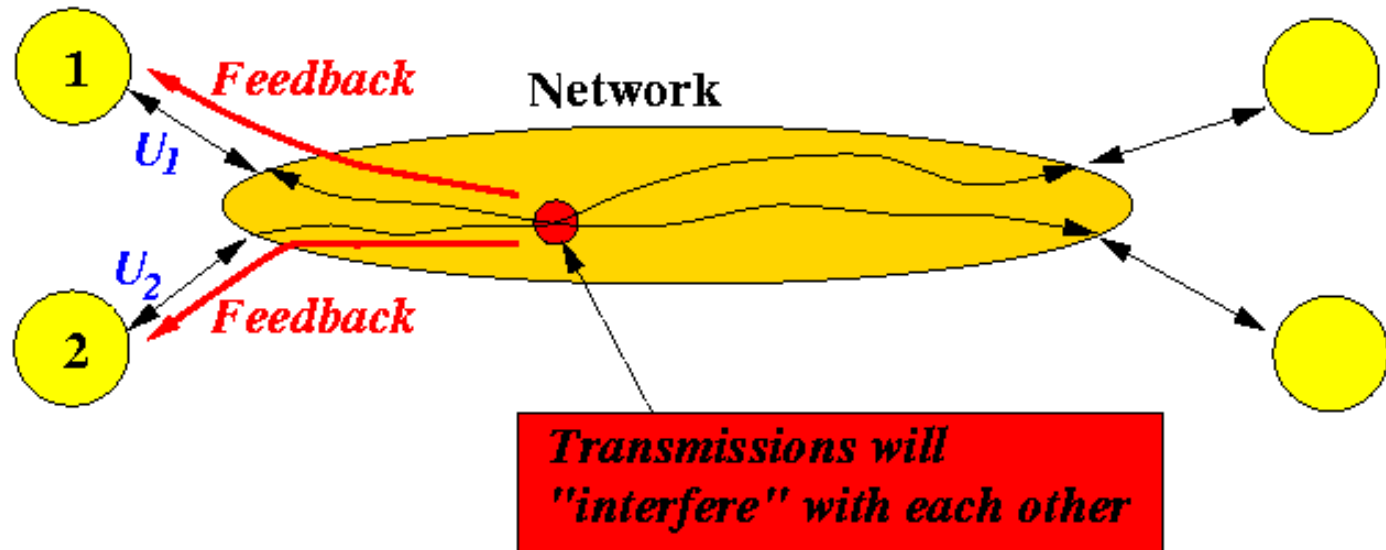
- *Vector Representation*

- Resource allocation for n users can be represented as a point in an n - dimensional vector space.
- For n = 2 users 2 - dimensional vector space for simplicity
- Any two users resource allocation can be represented as a point in a 2D space as $(x_1(t), x_2(t))$
- 2D space
- Let x axis represents allocation to user 1, and y axis represents allocation to user 2



Analysis: Consider two connections -

- The behavior of the congestion control mechanism can be revealed clearly by studying the behavior of only **two connections**:



Consider two users – whose connection to their destinations are passing through the single router line / buffer

2D vector space

Each point in this 2D space represents the possible loads by two users

Let us consider
4 points

A, B, C, D

Where

A: $x < y$

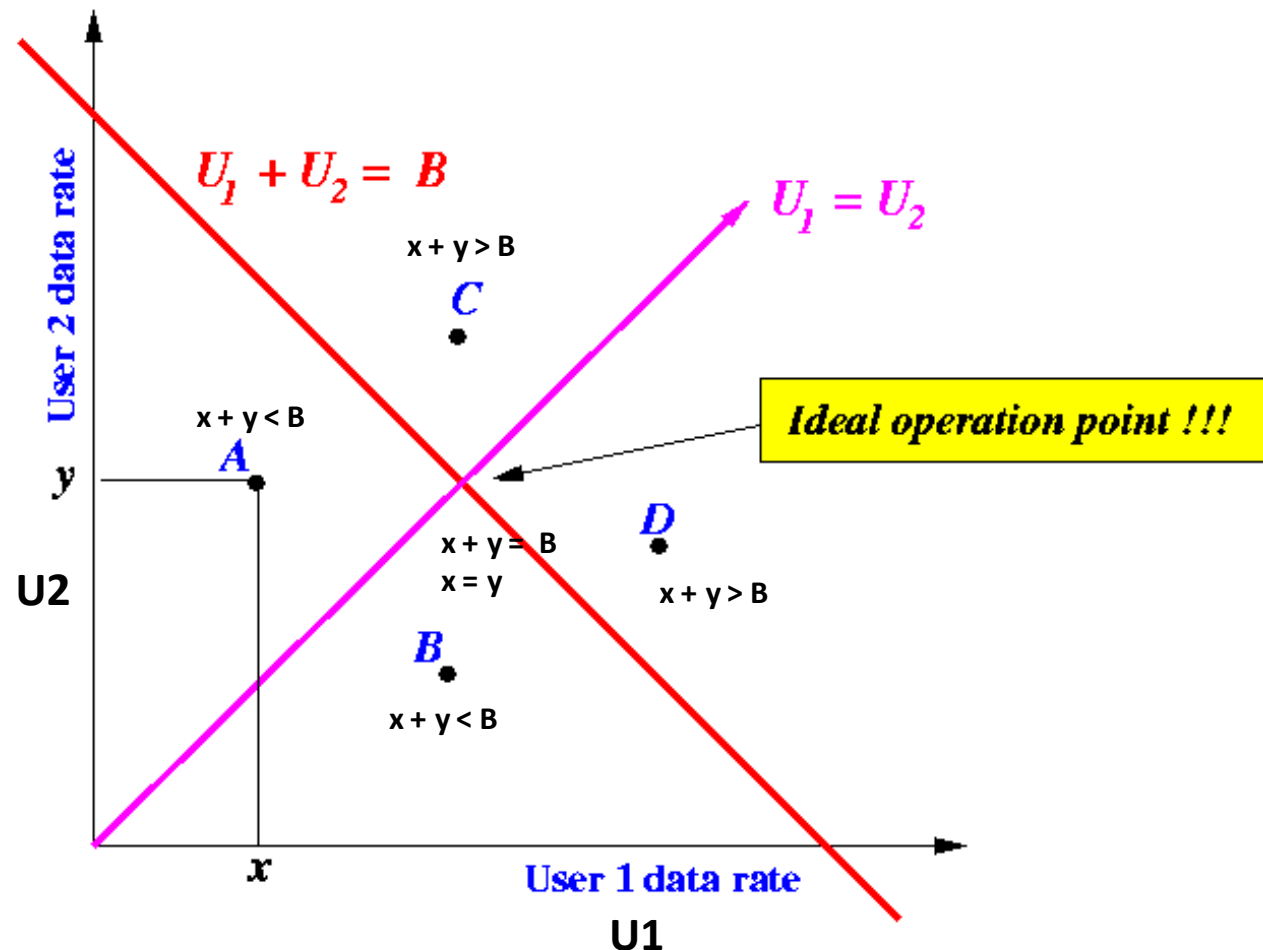
B: $x > y$

And $x + y < B$

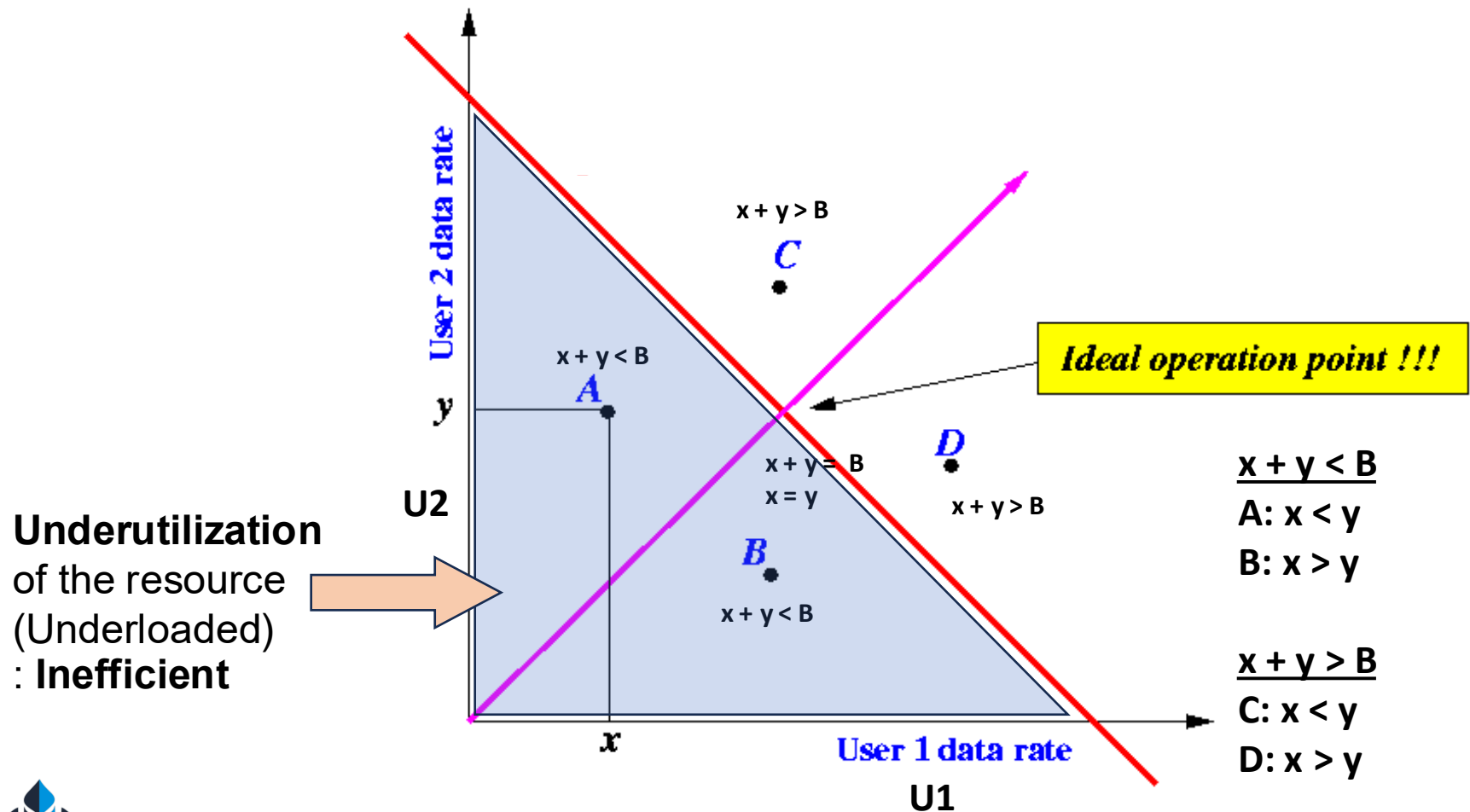
C: $x < y$

D: $x > y$

And $x + y > B$



2D vector space



2D vector space

$$x + y < B$$

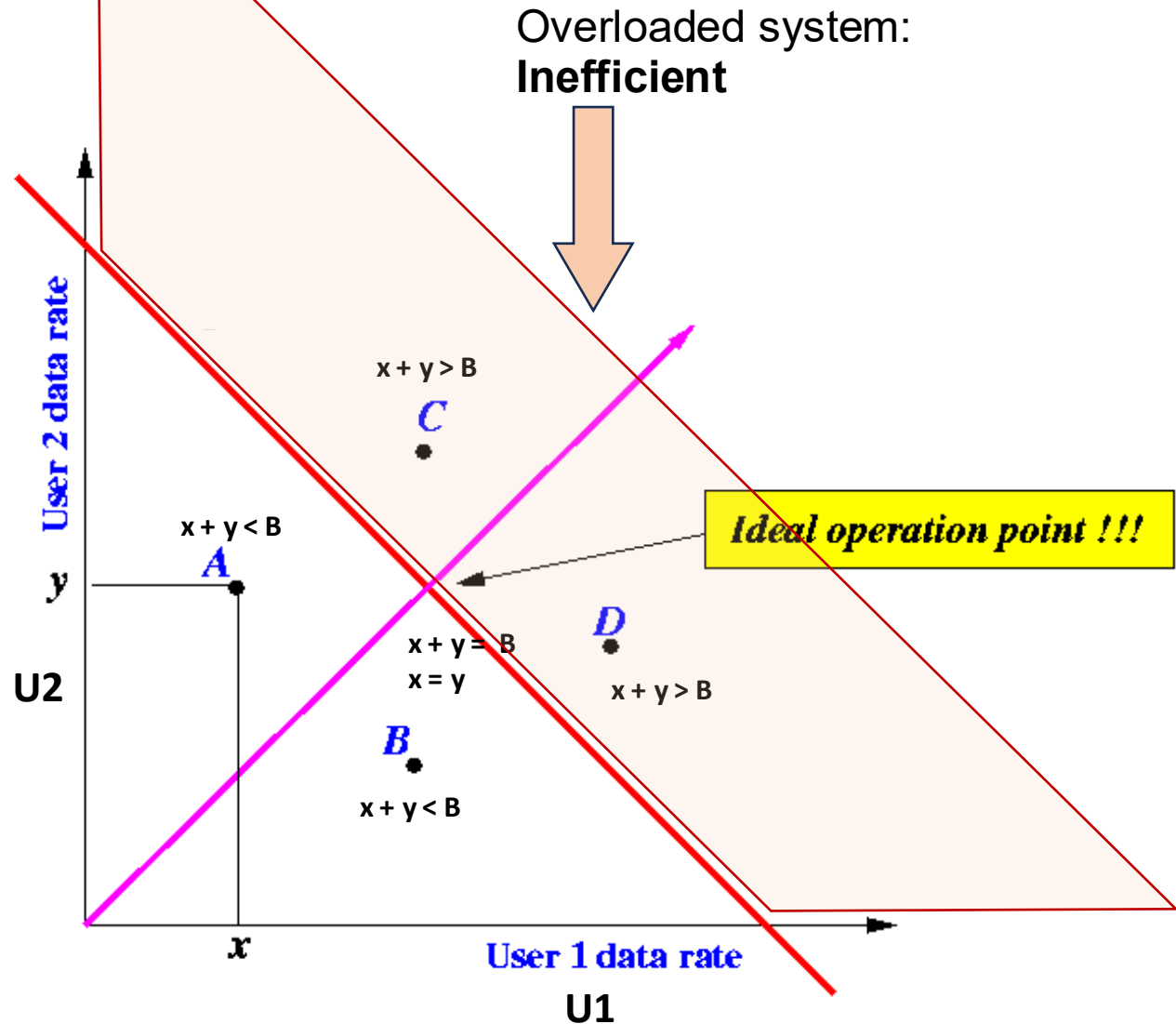
$$A: x < y$$

$$B: x > y$$

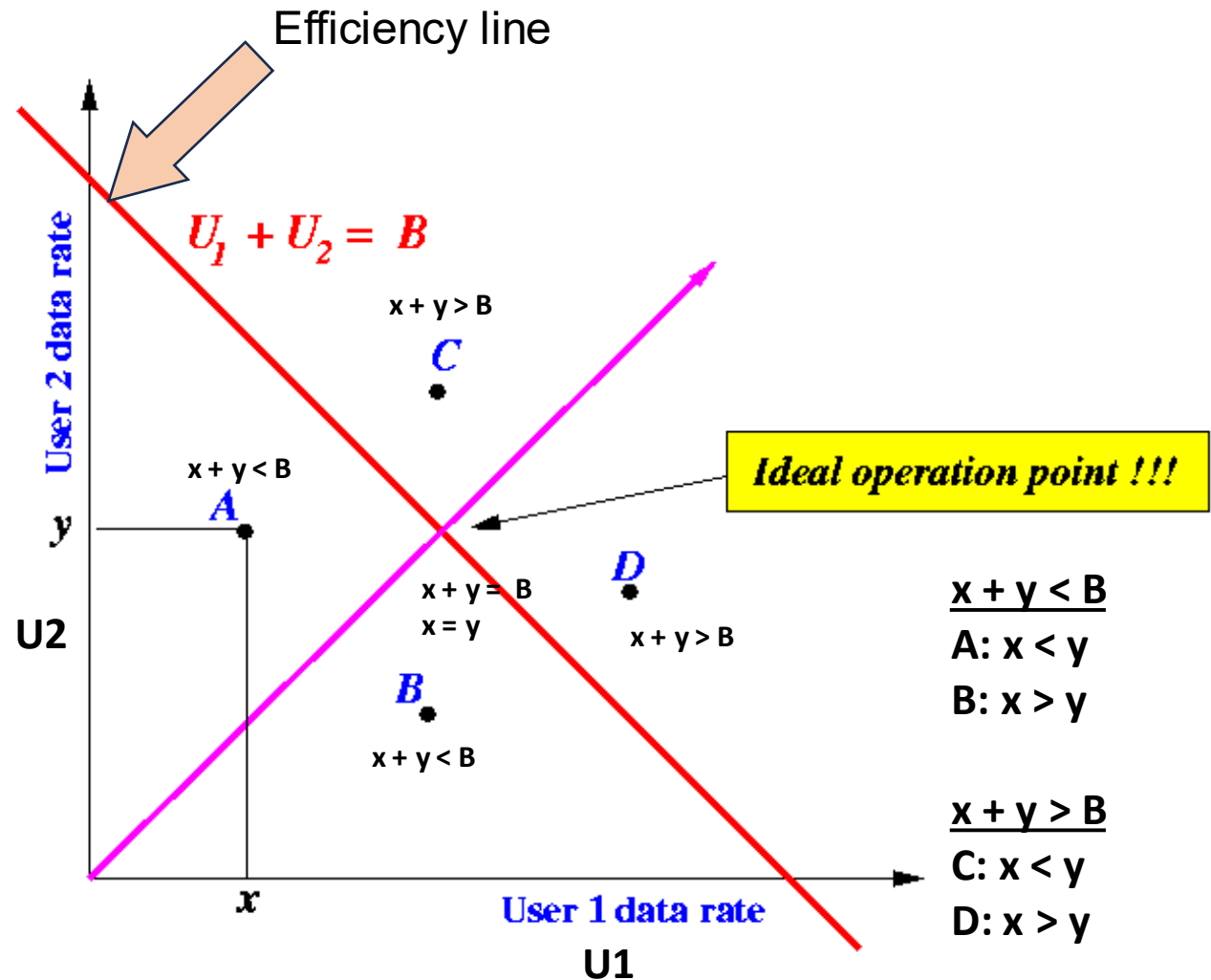
$$x + y > B$$

$$C: x < y$$

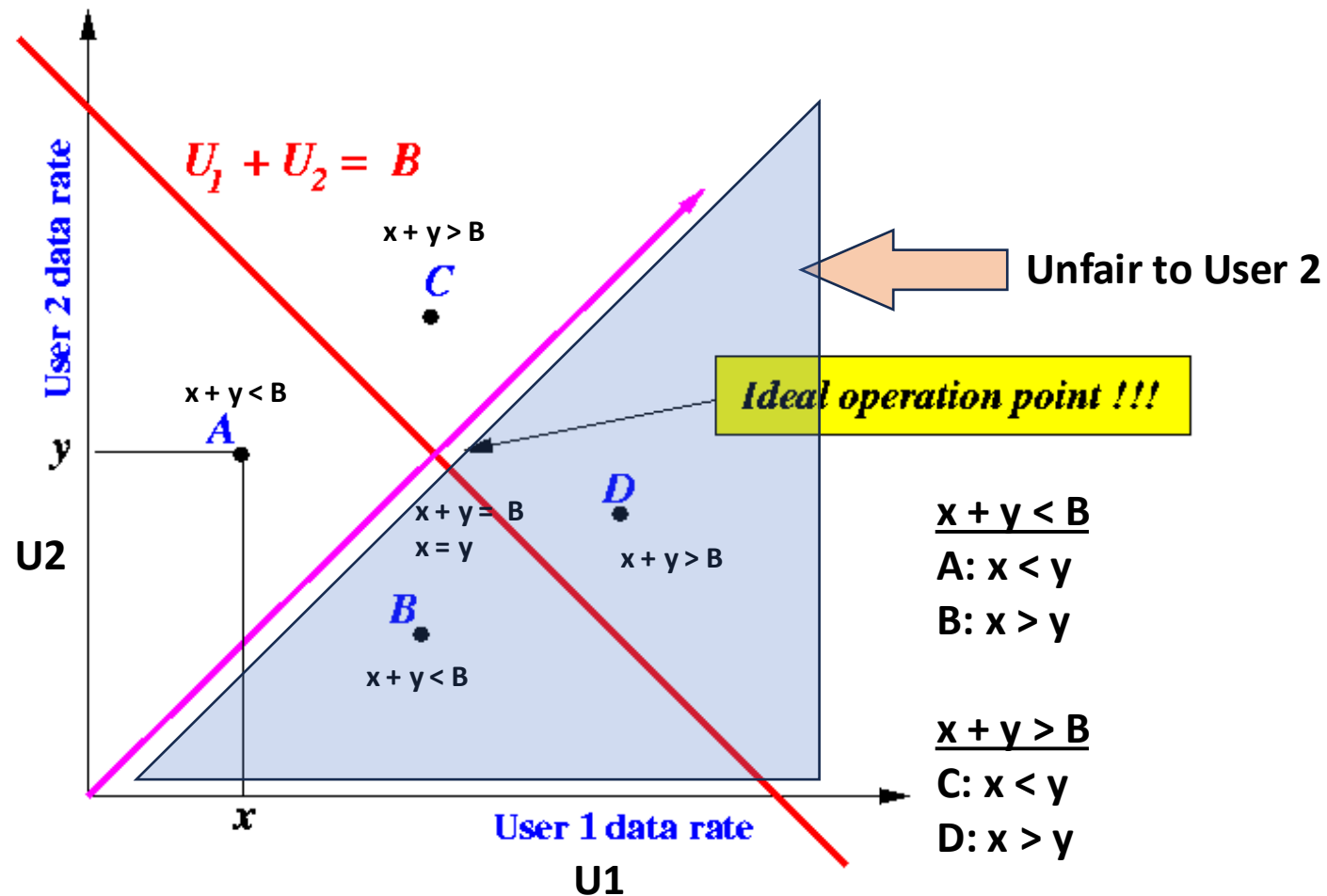
$$D: x > y$$



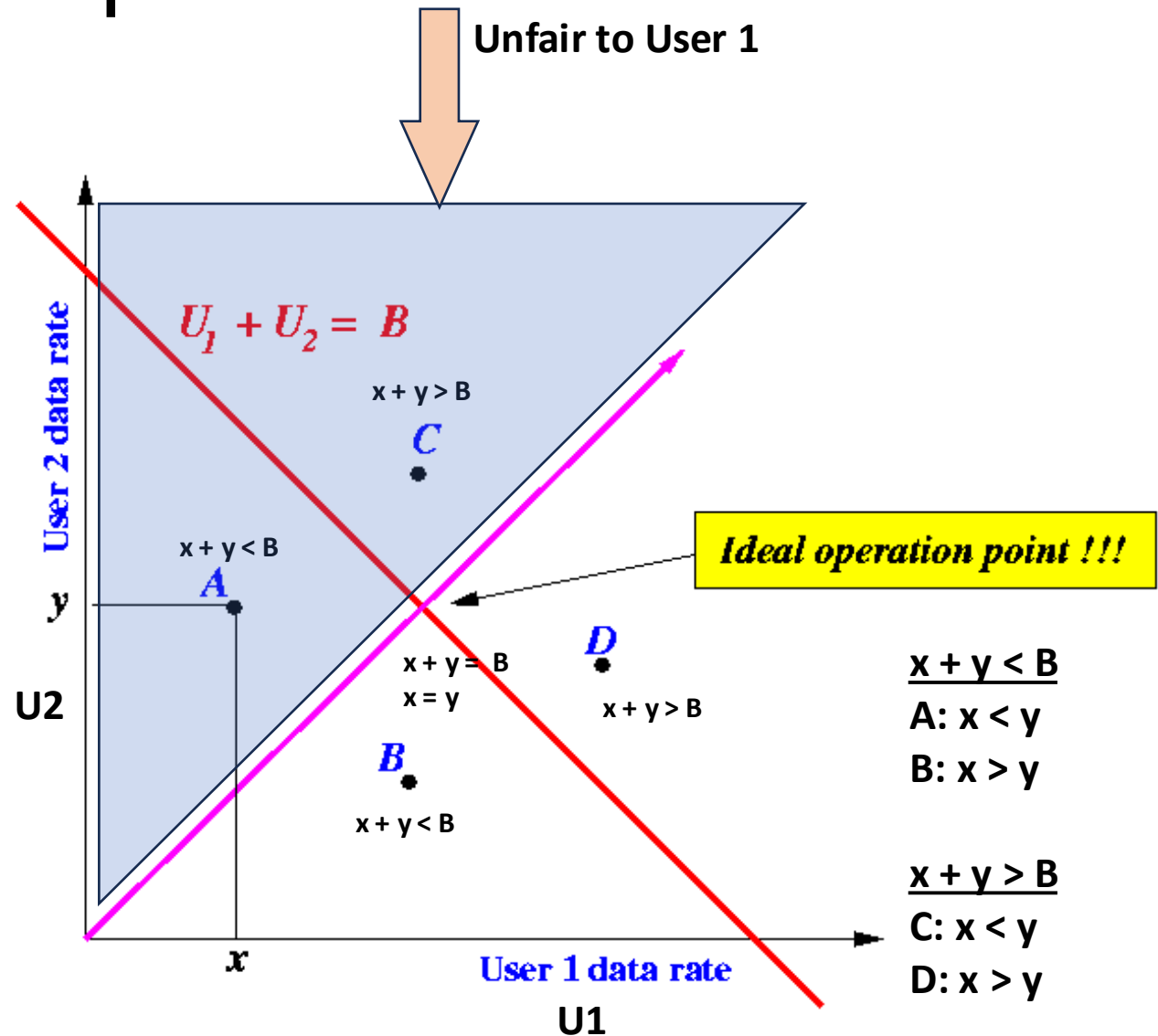
2D vector space



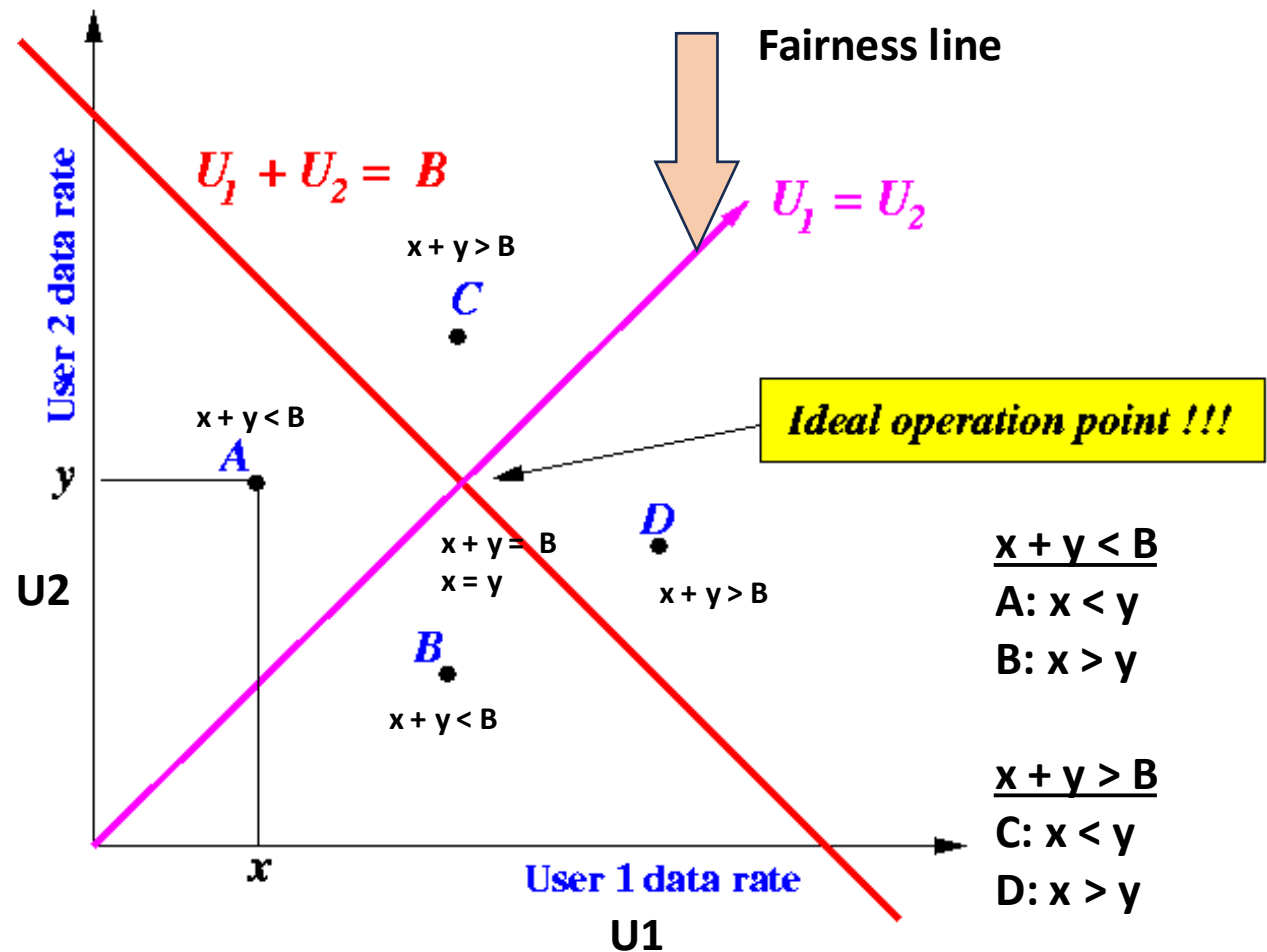
2D vector space



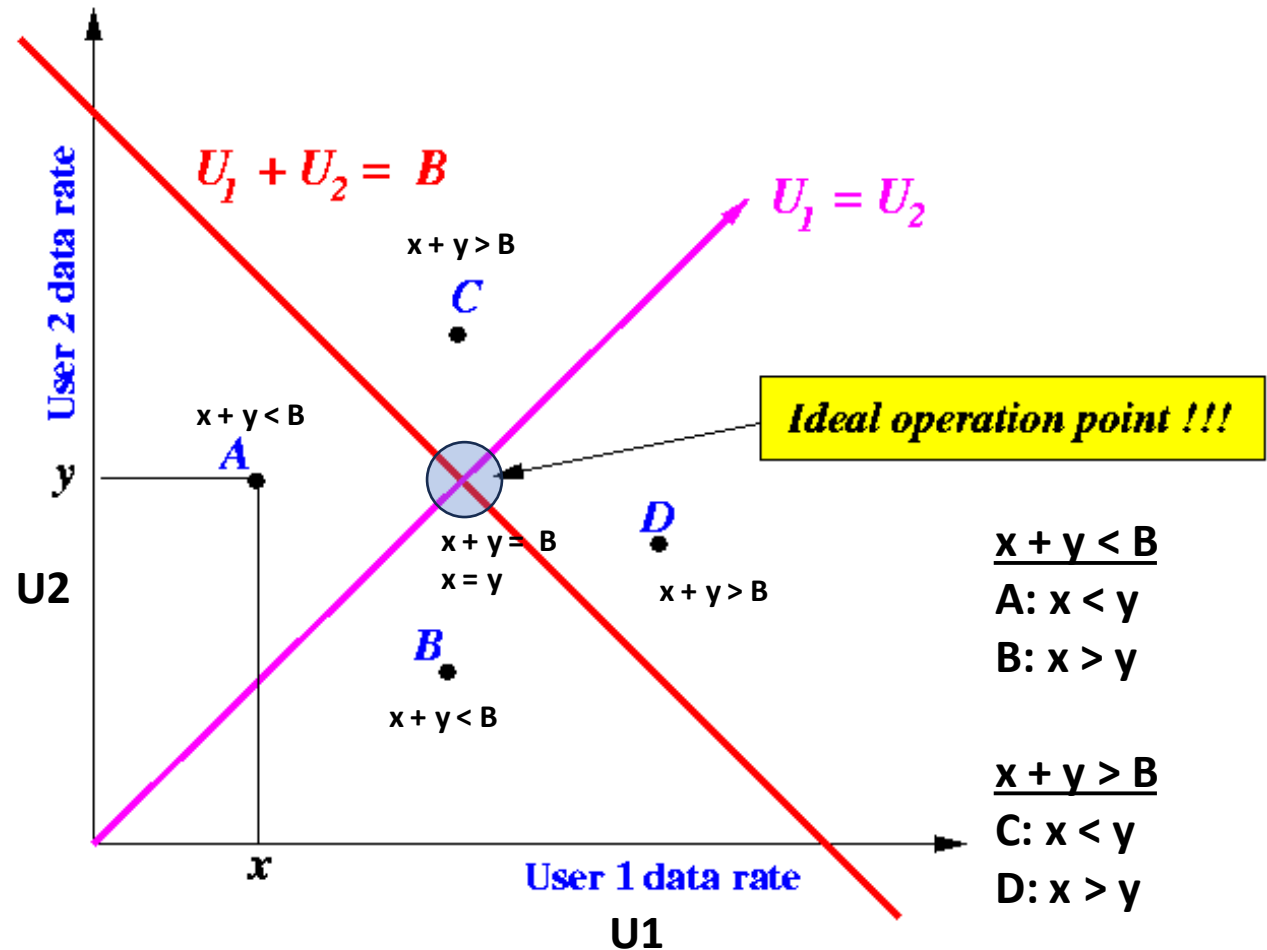
2D vector space



2D vector space



2D vector space



What about the metrics?

Efficiency

- All allocations for which $x_1 + x_2 = X_{\text{goal}}$ are efficient allocation
- A straight-line can represent that efficiency line $x + y = X_{\text{goal}}$

Fairness

- All allocations where $x_1 = x_2$ are **fair allocations**
- We can have a fairness line in the graph 2D space

Fairness + Efficiency

- The point where the two lines intersect is **fair + efficient** point
- The allocation is $x_1 = X_{\text{goal}} / 2$ and $x_2 = X_{\text{goal}} / 2$
- The goal of any control scheme is to bring the system to this point **regardless of the starting point**



Next ...

Starting from any given point in 2D space

How to reach the optimal point quickly?

How different functions behave?

Behavior of different linear functions ...

Additive increase / Decrease

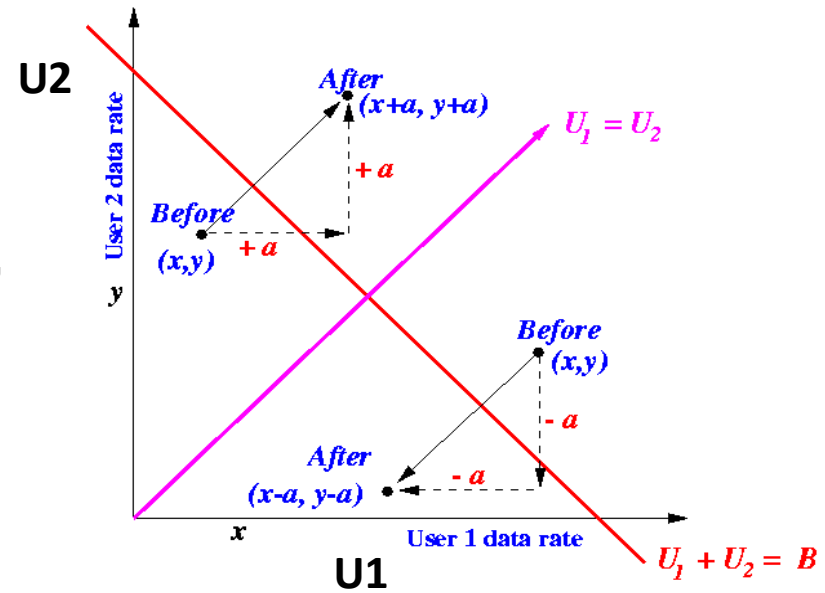
Increase $x \leftarrow x + a_I, y \leftarrow y + a_I$

Decrease $x \leftarrow x - a_D, y \leftarrow y - a_D$

Does it affect fairness?

Compare $\rightarrow x/y$ and $(x+a)/(y+a)$

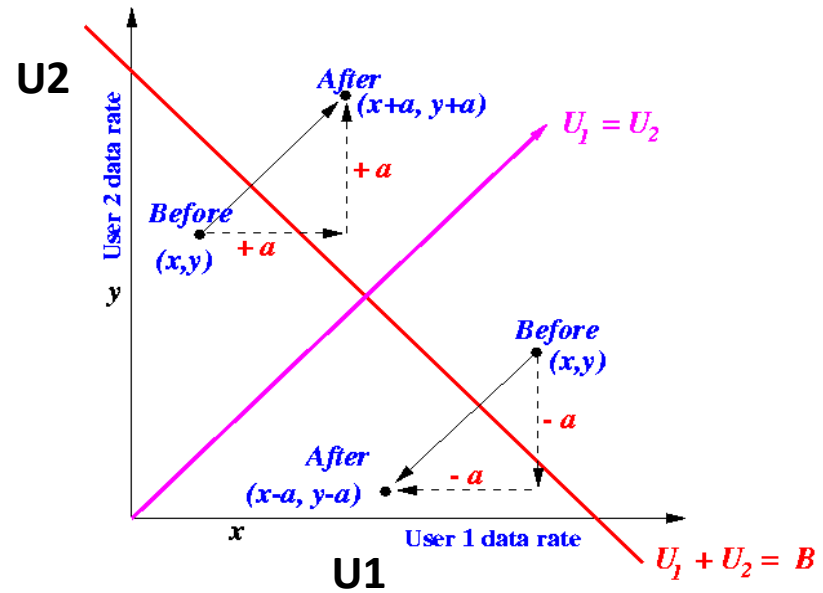
If $x/y > 1$ then $(x+a)/(y+a)$ will tend to make it close to 1 as a increases



So what happens to the fairness if you move through the line?

Behavior of different linear functions ...

Does it affect fairness?



Example: Compare $2/7$ and $(2+5)/(7+5)$ and $(2+21)/(7+21)$

If $x/y > 1$ then $(x+a)/(y+a)$ will tend to make it close to 1 as a increases

So what happens to the fairness if you move through the line?

Properties

Multiplicative Increase/ Decrease:

$$x \leftarrow x * a_i$$

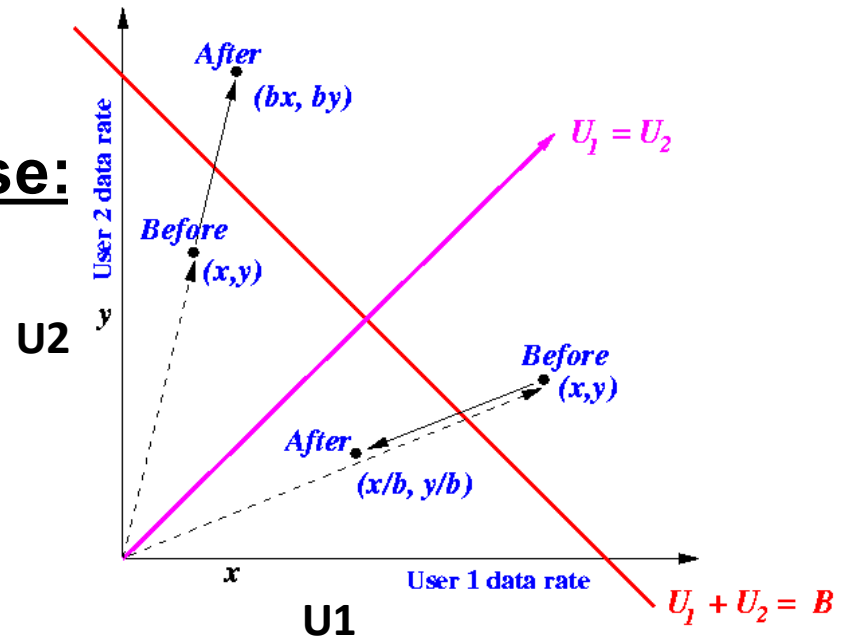
$$y \leftarrow y * a_i$$

Does it change fairness?

Compare $\rightarrow x/y$ and $(x*a)/(y*a) \rightarrow$ **No change in fairness.**

*It implies moving up or down through a line that connects the point with the origin- **it does not change fairness***

They are called **Equi-Fairness** lines

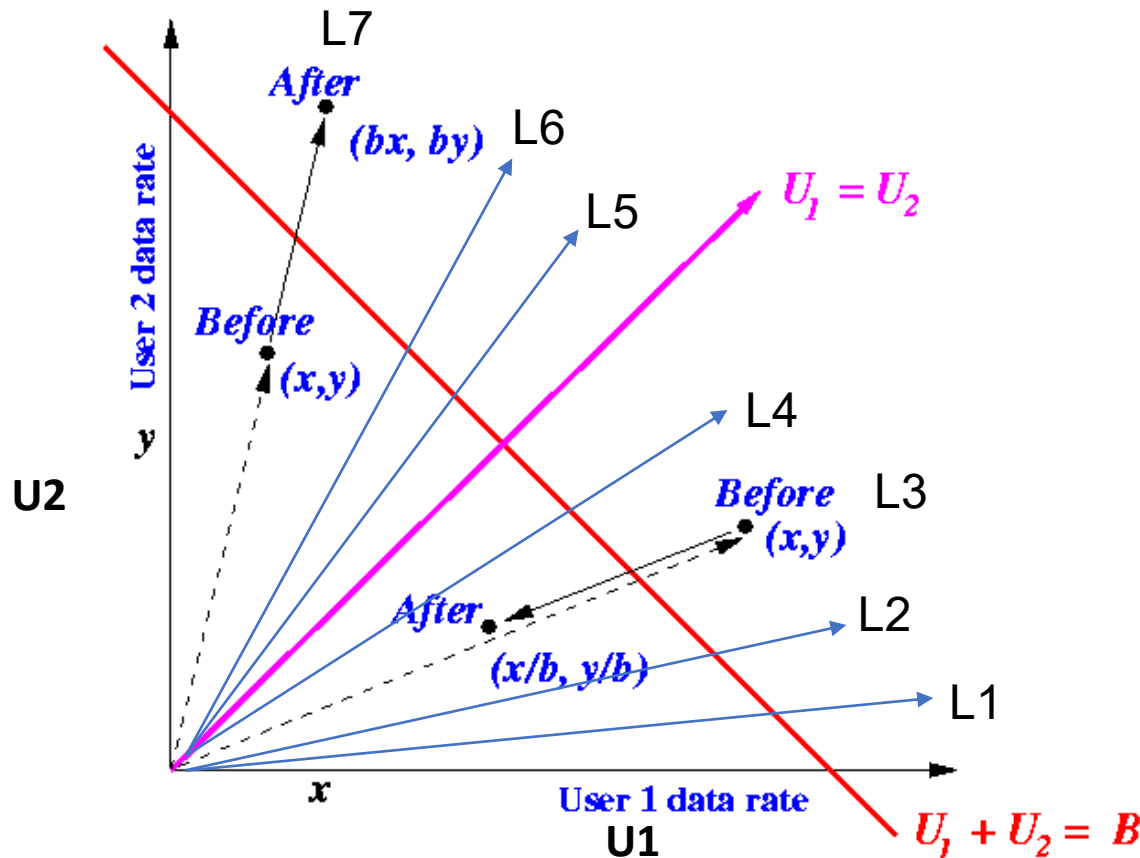


Fairness (Generalized discussion)

- Recall: Fairness is calculated by: $F = \frac{(\sum x(t))^2}{n \sum (x(t))^2}$
- For a two users setting it comes as: $\frac{(x_1 + x_2)^2}{2(x_1^2 + x_2^2)}$
- Adding a component 'a' with both the x_1 and x_2 changes fairness?
- [Check for both +, - and *, /]
- **Multiplying** both the x_1 and x_2 by a factor does not change the fairness (bx_1, bx_2) for any value of b: **Equi-fairness line**
- Ideally Fairness goes less as the slope of the line increases above or decreases below the slope of the Fairness line



Properties



Moving along a line ($L_1, \dots, L_7$) does not change fairness

But L_5 and L_4 have the highest fairness

Compared to the other lines

In Summary:

- 1) $L_7 < L_6 < L_5$
- 2) $L_4 > L_2 > L_1$

Properties

So what should be the strategy?

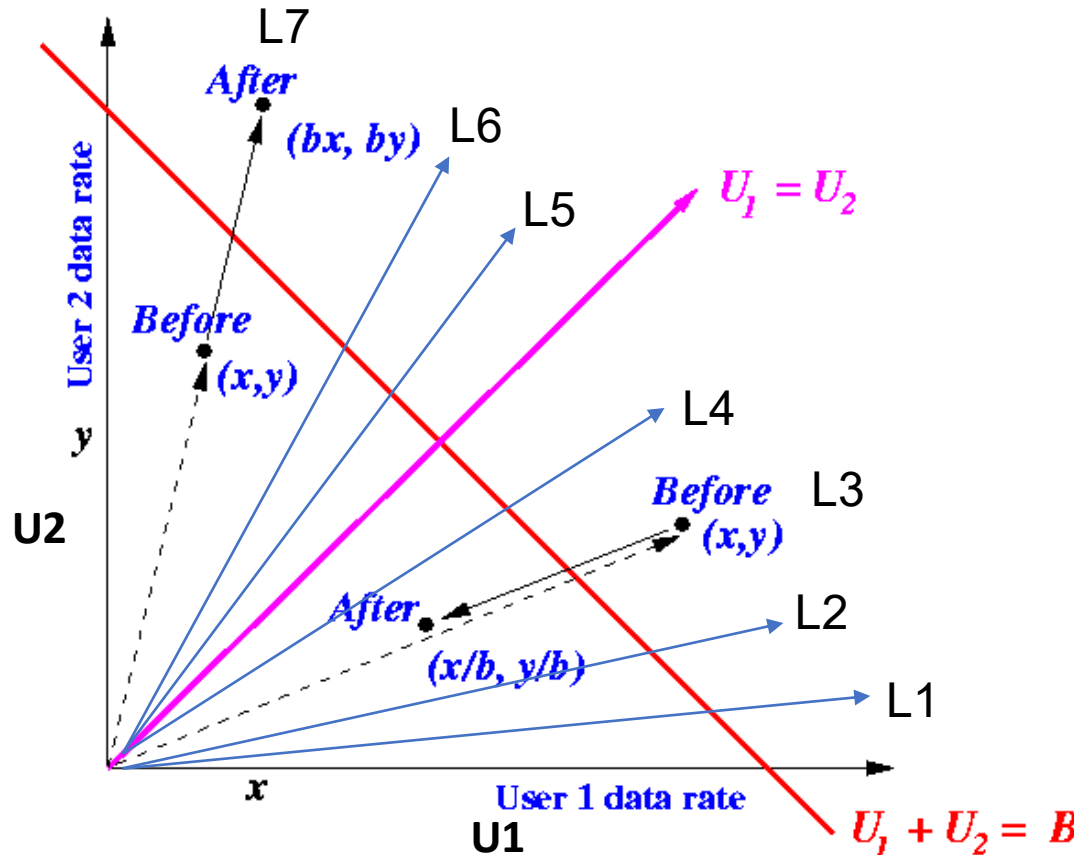
One possibility:

Use **additive** to go above $x+y = B$
→ Attempt to make it efficient

But ultimately make it **inefficient**

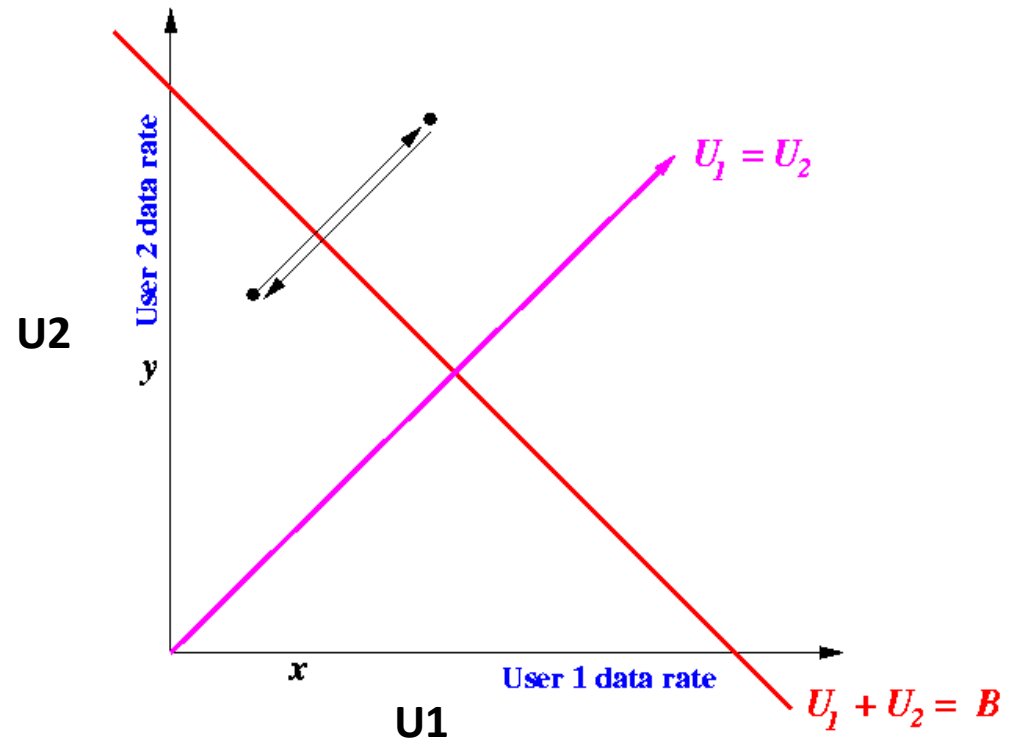
There will be a change in the fairness

So decrease using **multiplicative** – while keeping fairness same



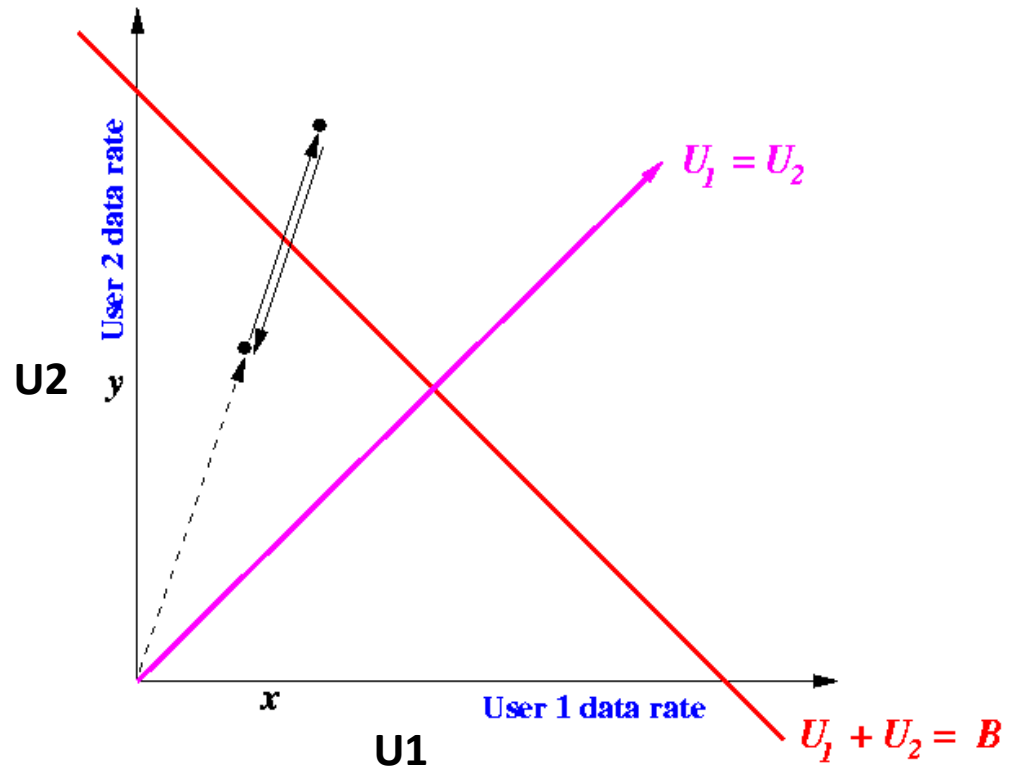
Possible strategies

- **Additive Increase and Additive Decrease (AIAD)**
- Method does not converge/move



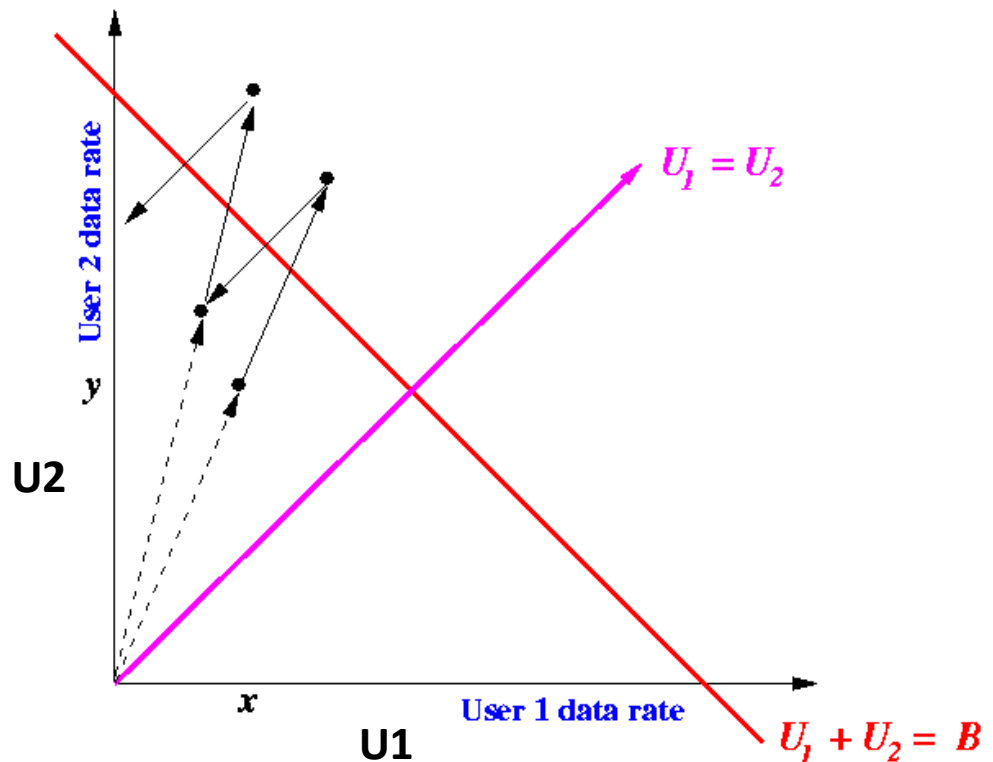
Possible strategies

- **Multiplicative Increase Multiplicate Decrease (MIMD)**
- Does not converge/move



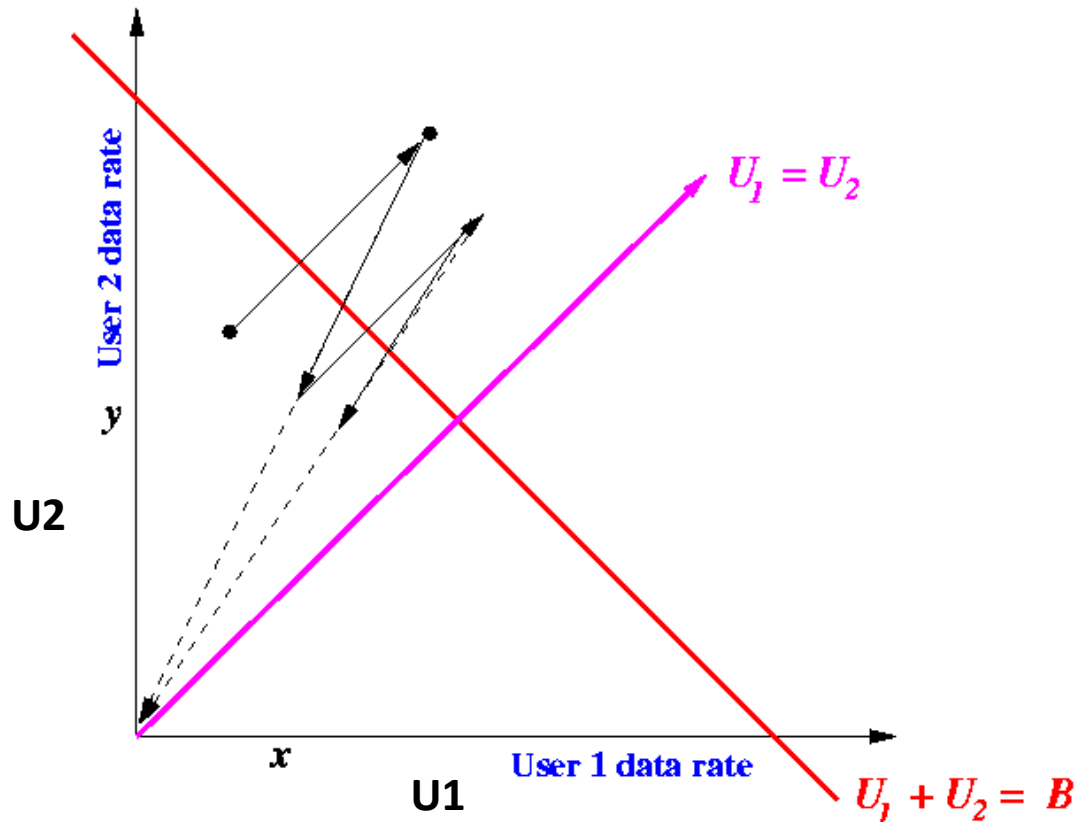
Possible strategies

- **Multiplicative Increase and Additive Decrease**
- It diverges away from the optimal point

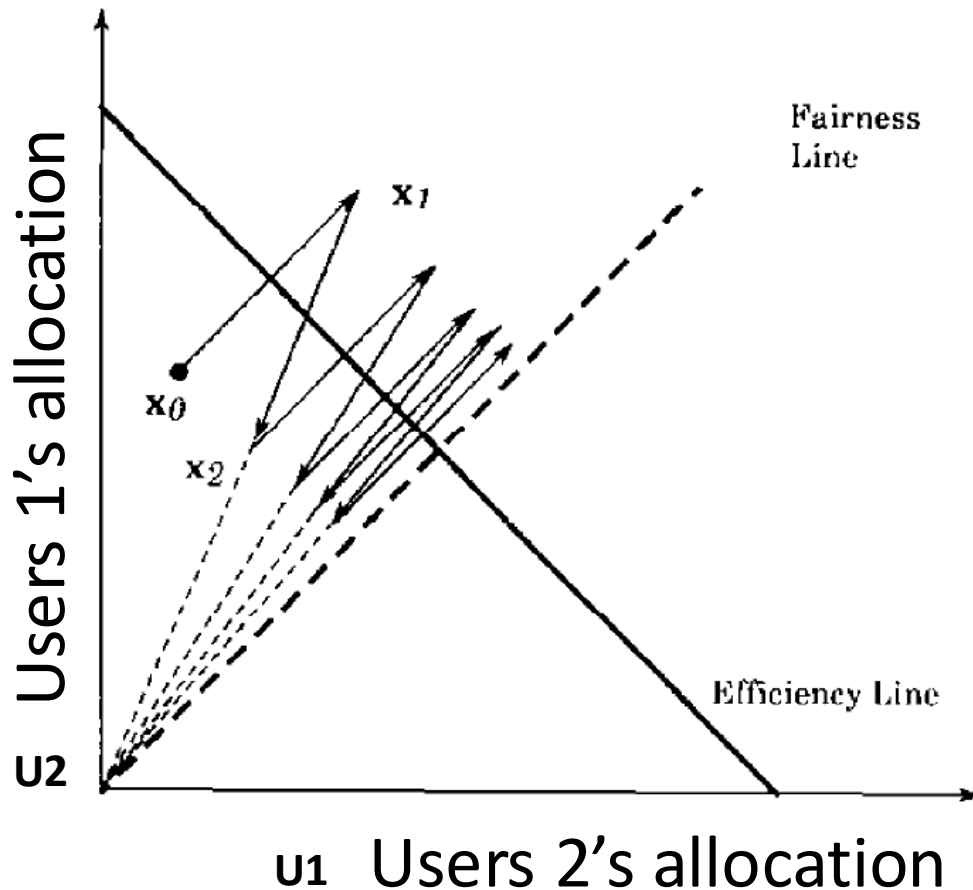


Possible strategies

- Additive Increase Multiplicative Decrease
- Converges towards the optimal point



AIMD (Additive Increase and Multiplicative Decrease)



(1) Initial Point starts with is x_0

- Below Efficiency Line
- Increase the load – *Both users*

(2) Additive Increase

- Move along the 45 degree line
- Comes to x_1 – above efficiency line

(3) Multiplicative decrease

- Moving towards the origin
- Comes at x_2

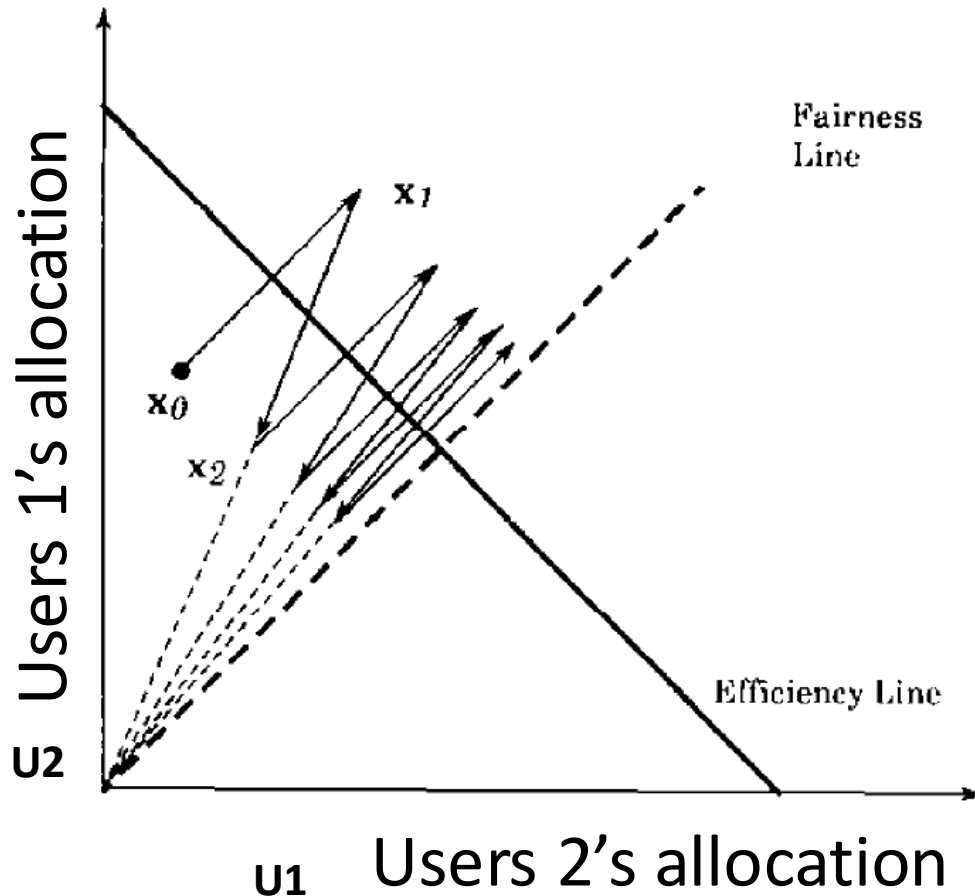
(4) Below efficiency, so additive increase again

- Continues this way...

(5) Eventually –

Oscillation over the optimal point (here additive increase and multiplicative decrease cause same slope)

AIMD (Additive Increase and Multiplicative Decrease)



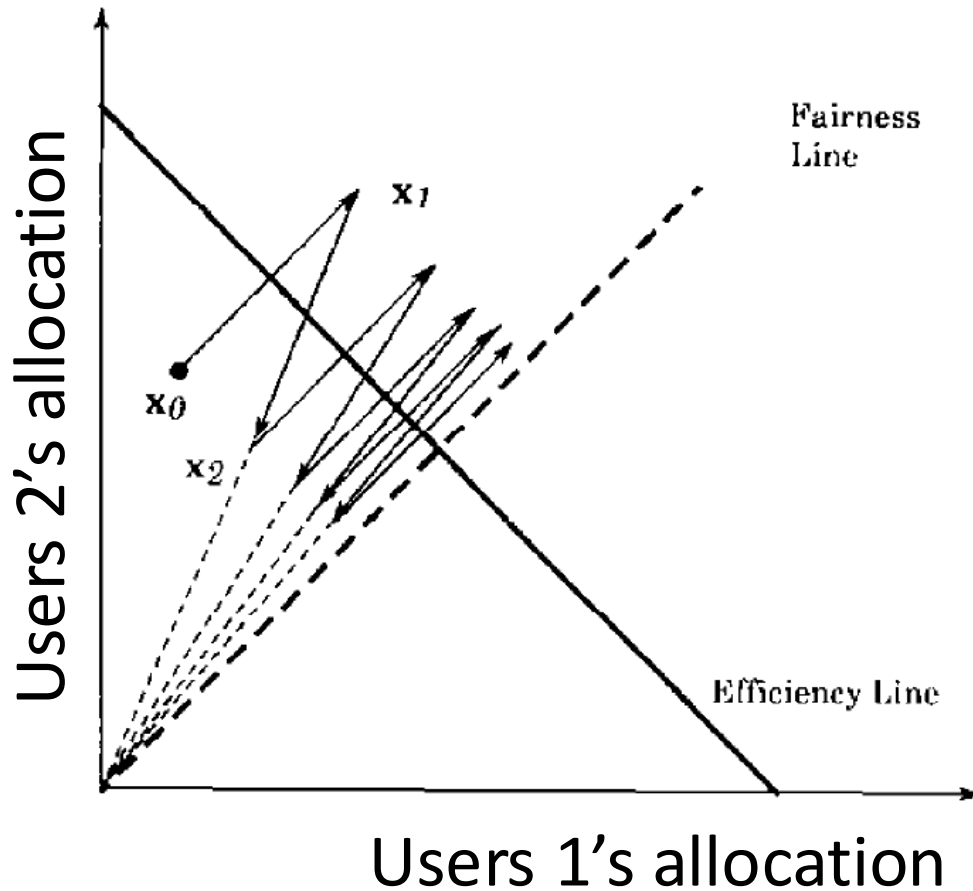
Additive Increase

- Increases the use by the users
- But trying to make it more fair

Multiplicative Decrease

- Decreases the use
- That is creates scope of increase in user in the next step -
- Keeping the fairness same

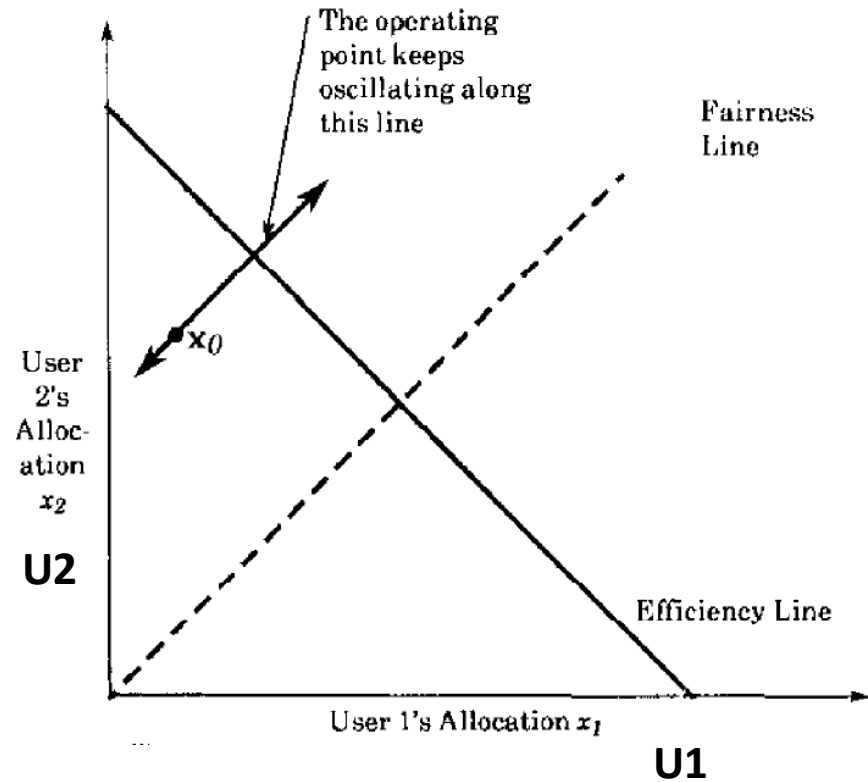
AIMD (Additive Increase and Multiplicative Decrease)



- One cycle:
- ***Additive Increase and then Multiplicative decrease***
- With every cycle –
 - Fairness increases slightly
 - Eventually system converges to the optimal point
- **But the system oscillates around the goal**

Other possible strategies

- All control policies do not converge
- e.g. Additive Increase and Additive Decrease
- The system moves back and forth through 45 degree slope
- *It converges to efficiency but not to fairness*



What we need for convergence?

- (1) Network gives a feedback
- (2) The system should react in the right direction
- (3) So that we converge to efficiency and fairness both

Principle of negative feedback

- When Feedback is underloaded (0) –
- We need to ensure total load next is **NOT** getting decreased

- When Feedback is Overloaded (1) –
- We need to ensure that total load next is **NOT** going to be increased



Convergence to **efficiency**

y(t) is the feedback, therefore,

$$y(t) = 0 \Rightarrow \sum x_i(t+1) > \sum x_i(t),$$

$$y(t) = 1 \Rightarrow \sum x_i(t+1) < \sum x_i(t).$$

Recall the linear functions for increasing and decreasing load per user i

- **y = 0,** $x_i(t+1) = a_I + b_I \cdot x_i(t)$
- **y = 1,** $x_i(t+1) = a_D + b_D \cdot x_i(t)$

Satisfying the negative feedback principle we get some properties of these functions

(Apply Principle of negative feedback)

When **y = 0,**

$$\sum x_i(t+1) > \sum x_i(t)$$

$$\rightarrow \sum x_i(t+1) - \sum x_i(t) > 0$$

$$\rightarrow \sum (a_I + b_I \cdot x_i(t) - x_i(t)) > 0$$

$$\rightarrow \mathbf{n \cdot a_I + (b_I - 1) \sum x_i(t) > 0}$$

When **y = 1,**

$$\sum x_i(t+1) < \sum x_i(t)$$

$$\rightarrow \sum x_i(t+1) - \sum x_i(t) < 0$$

$$\rightarrow \sum (a_D + b_D \cdot x_i(t) - x_i(t)) < 0$$

$$\rightarrow \mathbf{n \cdot a_D + (b_D - 1) \sum x_i(t) < 0}$$



Convergence to **efficiency**

y(t) is the feedback, therefore,

When $y = 0$,

$$\sum x_i(t+1) > \sum x_i(t)$$

$$\rightarrow \sum x_i(t+1) - \sum x_i(t) > 0$$

$$\rightarrow \sum (a_I + b_I \cdot x_i(t) - x_i(t)) > 0$$

$$\rightarrow n \cdot a_I + (b_I - 1) \sum x_i(t) > 0$$

Therefore,

$$b_I > 1 - [n \cdot a_I / \sum x_i(t)]$$

When $y = 1$,

$$\sum x_i(t+1) < \sum x_i(t)$$

$$\rightarrow \sum x_i(t+1) - \sum x_i(t) < 0$$

$$\rightarrow \sum (a_D + b_D \cdot x_i(t) - x_i(t)) < 0$$

$$\rightarrow n \cdot a_D + (b_D - 1) \sum x_i(t) < 0$$

Therefore,

$$b_D < 1 - [n \cdot a_D / \sum x_i(t)]$$

Convergence to **efficiency**

- Let us see in AIMD what we got

When $y = 0$,

We have Additive Increase

$$x_i(t+1) = a_i + b_i \cdot x_i(t) \rightarrow b_i = 1$$

Therefore,

$$\rightarrow 1 > 1 - n \cdot a_i / \sum x_i(t)$$

$$\rightarrow a_i / \sum x_i(t) > 0$$

$$\rightarrow a_i > 0$$

$$y = 0 \rightarrow x_i(t+1) = a_i + x_i(t)$$

$$y = 1 \rightarrow x_i(t+1) = b_D \cdot x_i(t)$$

When $y = 1$,

We have multiplicative decrease

$$x_i(t+1) = a_D + b_D \cdot x_i(t) \rightarrow a_D = 0$$

Therefore,

$$\rightarrow b_D < 1 - [n \cdot a_D / \sum x_i(t)]$$

$$\rightarrow b_D < 1$$



Convergence to **Fairness**

This implies moving towards fairness index of 1 ($F(x(t)) \rightarrow 1$ as t increases)

Apply on the functions -

$x_i(t+1) = a + b x_i(t)$ [for both inc and dec]

$$F(x(t+1)) = \frac{[\sum x_i(t+1)]^2}{n \cdot \sum x_i^2(t+1)}$$

$$= \frac{[\sum a_i + b_i \cdot x_i(t)]^2}{n \cdot \sum [a_i + b_i \cdot x_i(t)]^2}$$

$$= \frac{[\sum c + x_i(t)]^2}{n \cdot \sum [c + x_i(t)]^2}$$

$$c = a_i/b_i$$

$$\rightarrow \left(\frac{F(x(t)) + (1-F(x(t)))^*}{1 - \frac{\sum x_i^2(t)}{\sum (c + x_i(t))^2}} \right)$$

- Last part is an **increasing function of c**
- If $c > 0 \rightarrow F(x(t+1)) = F(x(t)) + \text{[+Ve no.]}$
- If $c = 0 \rightarrow F(x(t+1)) = F(x(t))$
- **$\rightarrow$ No change in Fairness**



Convergence to Fairness

- It is sufficient to ensure $c \geq 0$ for non-decreasing fairness
- $c = 0$ implies no change in fairness
- So we need $c > 0$ – in terms of parameters we need the following
- c is in both **increase** and **decrease** → **at least** one should imply increase in fairness

Possibility 1

$$a_i / b_i \geq 0 \text{ and } a_D / b_D > 0$$

Fairness goes up during decrease

Fairness goes up or stays same during increase

Possibility 2

$$a_i / b_i > 0 \text{ and } a_D / b_D \geq 0$$

Fairness goes up during increase

Fairness goes up or stays same during decrease

We need fairness to go up only during either increase or decrease, Not both



All are positive

In Summary

$$a_l / b_l \geq 0 \text{ and } a_D / b_D > 0$$

$$a_l / b_l > 0 \text{ and } a_D / b_D \geq 0$$

These implies - a_l , b_l , and a_D , b_D cannot be of opposite sign

Thus, both negative or both positive

Both cant be negative → So both are positive.

Thus → a_l , b_l , and a_D , b_D → all are positive...



Characteristics of the constants...

So, a_I , b_I , and a_D , $b_D \rightarrow$ all are positive

We already know $b_D < 1 - [n \cdot a_D / \sum x_i(t)]$ [From efficiency formula]

Here n , $x_i(t)$ are positive. So $b_D < 1$

Thus we get the guidelines as follows –

$$\begin{aligned} a_I &> 0 \\ b_I &> 0 \\ a_D &> 0 \\ 0 &\leq b_D < 1 \end{aligned}$$

a_I and b_I cannot be both zero –

otherwise there will be zero increase

a_I and a_D cannot be both zero –

otherwise c is always zero $\rightarrow$ no change in fairness

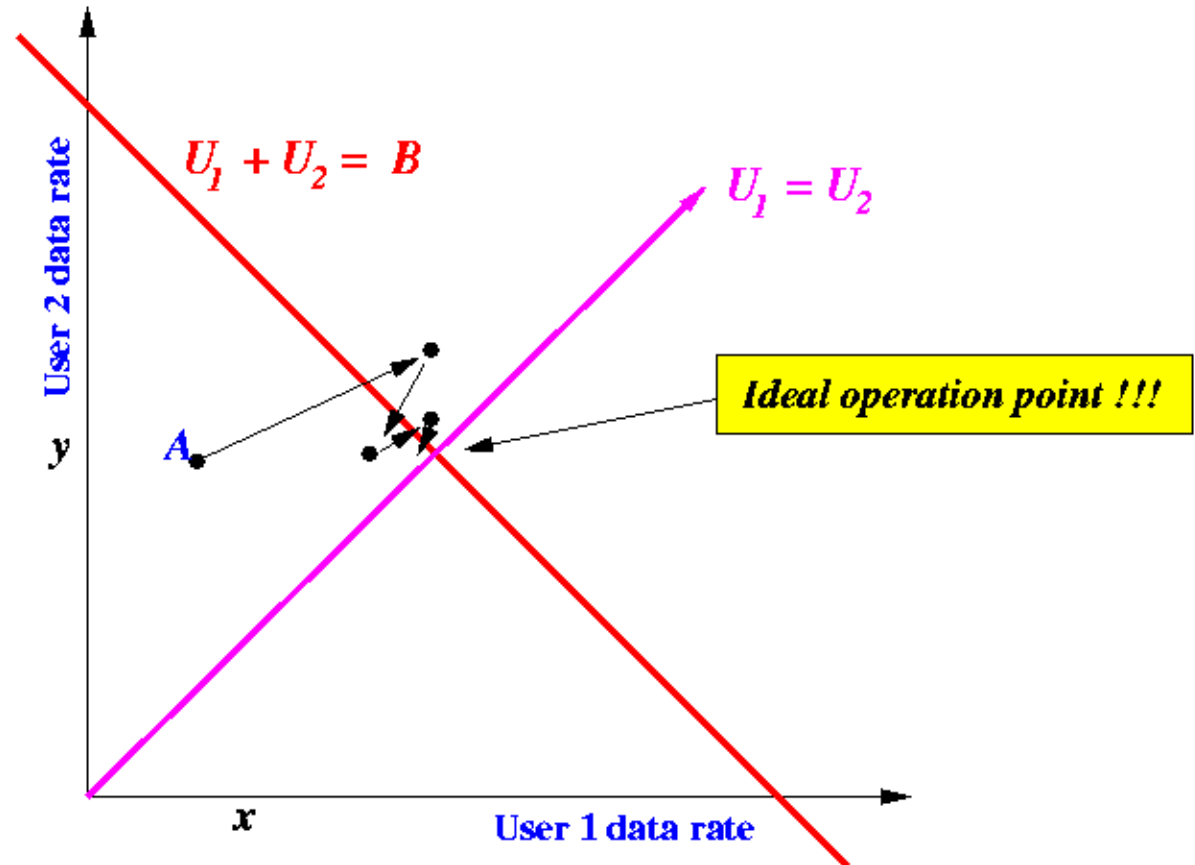


A possible ideal congestion control method

- The following figure shows the behavior of the **ideal** congestion control method:

Expectation

How?



Conclusion

- The **additive increase** and **multiplicative decrease** scheme will move the system towards a **stable and fair** equilibrium.
- Raj Jain's theory has held up for over 20 years.
- In those days, network speed were around 100 Kbps, at most 1 Mbps.
- Things has changed quite a bit, we have networks with speeds above 1 Tbps (1,000,000,000,000 bps).



Advanced methods ...

- In high speed networks, **additive increase** is **TOO SLOW** to allow the sender to fully utilize all the network capacity.
- Recent network research is proposing to use **multiplicative increase** and **multiplicative decrease** to control congestion.
- Hint: The factor used in **multiplicative decrease** is **different (it is larger)** than the one used in the **multiplicative increase**

